

CSE 232 : Data Structures

LAB 01 : ARRAY

Due Saturday at 5:00 p.m. MST

The first programming assignment of this semester is to write a class with the same functionality as the standard template library (STL) array class. As you may recall, the `std::array` class behaves much like the built-in array except it is a class and has member functions. The following is an example of the array type in action:

```
#include <array>    // std::array is declared here
#include <iostream> // for std::cout

int main()
{
    // Declare an array of 10 integers.
    std::array<int, 10> numbers;

    // Set the 4th element to 42. Indices start at 0.
    numbers[3] = 42;

    // Display the number of elements using the size() method.
    std::cout << "There are " << numbers.size() << " elements.\n";

    return 0;
}
```

To see a demo of how this works, please watch this video.

To learn more about the array data type, please review the following documentation: [std::array](#).

Array

Your array class will need to be defined in its own header file (`array.h`). The class name must be `array` (or, more accurately, `custom::array`). It will need to support the following operations:

- **Size:** Return the number of elements in the array. For the 85 option, this will always return 4. The other two options will return the template variable `N`.
See: [std::array::size](#).
- **Empty:** Return whether the array is empty. That is, whether the size is zero.
See: [std::array::empty](#).
- **Subscript:** Returns the i^{th} element in the array. There are two methods here. One returns a reference, the second returns a constant reference and the method is constant. No error checking is performed.
See: [std::array::operator\[\]](#).
- **At:** Returns the i^{th} element in the array. As with the subscript operator, there are two methods here. One returns a reference, the second returns a constant reference and the method itself is constant. Unlike the subscript operator, error checking is performed. If the client attempts to access an element that does not exist with a negative index or with an index that is greater than or equal to the size, then the following exception is thrown:
`std::out_of_range("ARRAY: index out of range")`.
See: [std::array::at](#).
- **Front:** Returns the first element in the array. This is always the same thing as `at(0)`. As with the subscript operator and the `at()` method, there are two versions: constant and

non-constant.

See: `std::array::front`.

- **Back:** Returns the last element in the array. This is always the same thing as `at(size()-1)`. As with the `front()` method, there are two versions: constant and non-constant.
See: `std::array::back`.
- **Begin:** Returns an iterator pointing to the first element in the array. Only return a non-constant iterator; we will not be creating a constant iterator.
See: `std::array::begin`.
- **End:** Returns an iterator pointing to the first element off the end of the array. Only return a non-constant iterator; we will not be creating a constant iterator.
See: `std::array::end`.

Note: the member variable must be called `__elems_`.

Iterator

Additionally, we will be creating an array iterator. This will exist in the same `array.h` that the array class is defined in. This class will have a pointer to an array element as the single member variable. The following methods will be supported:

- **Default constructor:** Sets the member variable pointer to `nullptr`.
- **Non-default constructor:** Accepts a pointer to an array element as a parameter and assigns that pointer to the member variable. Thus, if `p` points to the 2nd element in the array, then the iterator's member variable will point to the same element.
- **Copy constructor:** Copies the right-hand-side's member variable pointer to the member variable. Since there is exactly one member variable (the pointer to the element), then only the pointer will be copied.
- **Assignment operator:** Copies the right-hand-side's member variable pointer to the member variable. Again, like the copy constructor, this only copies the pointer.
- **Equals:** Returns `true` if the right-hand-side and left-hand-side both point to the same element. In other words, this is just a direct comparison of the member variable of the pasted iterator and that of the `this` iterator.
- **Not Equals:** Returns `false` if the right-hand-side and left-hand-side both point to the same element.
- **Dereference:** Returns the element that the iterator is pointing to. There are two variations. One returns the element by-reference. The second returns the element constant by-reference and is a constant method. In both cases, simply return dereferenced member variable.
- **Increment:** Advances the iterator so it points to the next element in the array. There are two variations. The first is the prefix increment which returns the iterator after the increment has occurred. The second is the postfix increment which returns a copy of the iterator before the increment has occurred. In both cases, simply increment the iterator's member variable.

Note: the member variable must be called `p`.

Three Options

You will have three options when implementing this class. These three options have different levels of difficulty and have a different potential score:

- **85:** The maximum score possible is 85%.

- **100:** The maximum score possible is 100%.
- **115:** The maximum score possible is 115%.

85 Option : Integer Fixed Size

This option will consist of implementing the array with all the above interfaces. The array will always have an integer as the base type. In other words, it will not be a template class. This implementation will always have exactly 4 elements. To declare an array, the following code is used:

```
array elements;
```

To complete the 85 option, please start with the following files and pass the tests contained therein: [232.01.Lab.85.zip](#).

[To see a demo of how this works, please watch this video.](#)

100 Option : Integer Variable Size

As with the 85 option, this option will consist of implementing the array with all the above interfaces and use an integer as the base type. Additionally, this implementation will be a template class where the client specifies the number of elements in the array. For example, the following code will instantiate an array of 5 elements:

```
array<5> elements;
```

To complete the 100 option, please start with the following files and pass the tests contained therein: [232.01.Lab.100.zip](#).

[To see a demo of how this works, please watch this video.](#)

115 Option : Template Base Type and Variable Size

This option will consist of implementing the array with all the above interfaces and will allow the client to specify the base type through a template parameter. This implementation will be a template class where the client specifies the number of elements in the array. For example, the following code will instantiate a string array of 5 elements:

```
array<std::string, 5> elements;
```

To complete the 115 option, please start with the following files and pass the tests contained therein: [232.01.Lab.115.zip](#).

The 115 option requires a precise number of allocations and deletions. In order to have more control over this, it is necessary to use an allocator. Please see Appendix I: Allocator in the text book for a detailed description of how this works. There is also a video demonstration here: [Allocators and Spys](#). This explains how the `spy` class works to count allocations, and how `std::allocator` can be used to more precisely control them.

[To see a demo of how this works, please watch this video.](#)

Unit Tests

Regardless of whether you attempt the 85, 100, or 115 option, the following will be provided:

- **Stubs:** The `array` class and the `array::iterator` class will be defined but with empty method definitions. This means that the code will compile but will not contain the necessary functionality.

- **Unit Tests:** Unit tests will be provided to thoroughly exercise your code. Initially, not all the unit tests will pass. To get the maximum possible score for your submission, each unit test must pass when you are finished.
- **Projects:** Whether you are using Microsoft Visual Studio or Apple X Code, projects are provided. If you are using Visual Studio, then double-click on the `labArray.sln` file. If you are using X Code, then double-click on the `labArray.xcodeproj` file.

Assignment

Your assignment is to create a working C++ program to match the program description. Please do the following:

1. **Install a code editor:** Install Visual Studio or X Code. Get a simple program to compile. You should have done this in the syllabus quiz earlier in the week.
2. **Select an option:** Select one of the three options: 85, 100, or 115. Note that if you select 85 and find it easier than you thought, you can download the 100 option and most of your code will transfer over.
3. **Download the code:** Download the `.zip` file and uncompress it on your computer. Run the code to make sure it works as you expect. Note that not all of the unit tests will pass.
4. **Implement the data structure:** Write the code to pass all the unit tests.
5. **Report on how it went:** Note how long you spend on the lab and comment on what was the hardest part.
6. **Upload your code:** Submit only `array.h` to I-Learn. Your instructor should be able to compile and run your code without modifying any of the other provided files.

Assessment

The grade for your assignment will be based on the following criteria:

	Exceptional 100%	Good 90%	Acceptable 70%	Developing 50%	Missing 0%
Code Quality 35%	The most elegant solution was found for all the methods; there is no room for improvement.	There are no errors in efficiency, maintainability, cohesion, robustness, or redundancy.	There exists one error in efficiency, maintainability, cohesion, robustness, or redundancy.	The code quality is not what is expected for a CSE 232 student.	The code is sloppy or otherwise unprofessional.
Functionality 50%	Passes all the tests	Passes 90% of unit tests	Passes 70% of unit tests	Passes 50% of unit tests	Program fails to compile or no file was uploaded.
Template Size 15%	The 100 or the 115 solution was attempted				The 85 solution was attempted
Template Base Type 15%	115 solution was attempted				The 85 or the 100 solution was attempted